

Documentación Técnica: Tarea 2 - Benchmark de Rendimiento

Desarrollador: Juan Camilo Lozano Cortés

Lenguaje: Python (Motor LALR vs. Algoritmo CYK)

1. Preparación del Entorno:

Se utilizó **Kali Linux** para ejecutar un script de pruebas de estrés (*benchmark*). Se empleó la librería Lark configurada con el motor **LALR** (algoritmo estándar de **Bison** y **ANTLR**) para compararlo contra una implementación manual del algoritmo **CYK**. El objetivo es medir el tiempo de respuesta frente a cadenas de entrada con longitud creciente (de 10 a 150 tokens).

2. Definición de Algoritmos y Simplicidad de Gramáticas:

En esta fase se evaluó la diferencia entre el análisis académico y el industrial:

- **Algoritmo CYK (Académico):** Implementado mediante programación dinámica. Aunque es universal para Gramáticas en Forma Normal de Chomsky, su eficiencia es de $O(n^3)$. Esto significa que el tiempo de procesamiento se dispara exponencialmente al aumentar el tamaño del código fuente.
- **Motor LALR (Industrial):** Representa herramientas como **Bison**. Su eficiencia es de $O(n)$ (lineal). Procesa el código de forma constante sin importar que el archivo sea extenso.

Justificación de Herramientas:

En lugar de programar cada parser desde cero (código mínimo), se optó por la **importación de gramáticas**. Esta metodología es superior porque:

1. **Abstracción:** Permite al desarrollador definir el lenguaje y dejar que el motor optimice la tabla de estados.
2. **Escalabilidad:** Es más sencillo y seguro importar una gramática probada que mantener miles de líneas de código manual para cada regla sintáctica.

3. Archivo de Pruebas y Generación de Cadenas:

```

4
5 # 1. Implementación del Algoritmo CYK ( $O(n^3)$ )
6 def cyk_parse(cadena, gramatica):
7     n = len(cadena)
8     if n == 0: return False
9     tabla = [[set() for _ in range(n)] for _ in range(n)]
10
11     # Paso 1: Terminales
12     for i in range(n):
13         for var, prods in gramatica.items():
14             if cadena[i] in prods:
15                 tabla[i][i].add(var)
16
17     # Paso 2: Producciones Binarias
18     for longitud in range(2, n + 1):
19         for i in range(n - longitud + 1):
20             j = i + longitud - 1
21             for k in range(i, j):
22                 for var, prods in gramatica.items():
23                     for p in prods:
24                         if isinstance(p, tuple) and p[0] in tabla[i][k] and p[1] in tabla[k+1][j]:
25                             tabla[i][j].add(var)
26     return "S" in tabla[0][n-1]
27
28 # Gramática en Formato Chomsky para CYK
29 gramatica_chomsky = {
30     "S": [("A", "B"), "a"],
31     "A": ["a"],
32     "B": [("0", "A"), ("0", "B")],
33     "0": ["+", "**"]
34 }
35
36 # 2. Configuración de Motor tipo BISON (LALR -  $O(n)$ )
37 grammar_bison = """
38 ?start: expr
39 ?expr: term ("+" | "**") term
40 ?term: "a"
41 %import common.WS
42 %ignore WS
43 """
44 parser_bison = Lark(grammar_bison, parser='lalr')
45
46 def benchmark():
47     # Probamos con cadenas de longitud creciente: 10, 50, 100, 200 tokens
48     escalas = [10, 50, 100, 150]
49
50     print(f'Tokens:<10 | {Tiempo CYK (s):<18} | {Tiempo BISON (s):<18}')

```

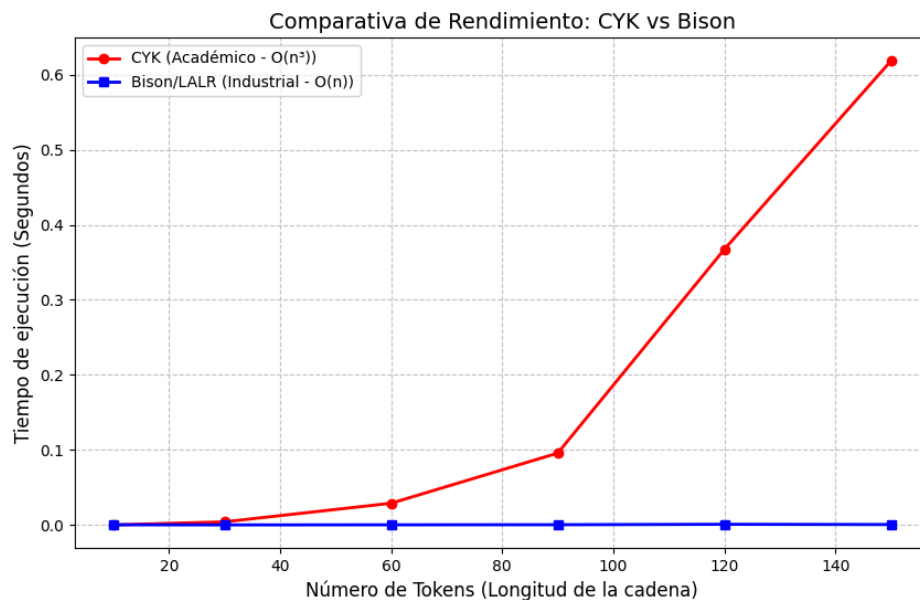
Para esta tarea no se usó un pruebas.txt estático, sino un generador dinámico en el código que crea expresiones del tipo $a + a * a \dots$ con longitudes de **10, 50, 100 y 150 tokens**. Esto permite observar el punto exacto donde el algoritmo CYK empieza a degradar el rendimiento del sistema.

4. Ejecución del Programa:

```
userx0@userx0: ~/TAREAS/Tarea2-Analisis_sintactico-main
Session Actions Edit View Help
(userx0@userx0)~$ cd /home/userx0/TAREAS/Tarea2-Analisis_sintactico-main/
(userx0@userx0)~/TAREAS/Tarea2-Analisis_sintactico-main$ python analizador_benchmark.py
Tokens | Tiempo CYK (s) | Tiempo BISON (s)
10 tokens de prueba | 0.000198 | 0.000341
50 tokens de prueba | 0.012926 | 0.000254
100 tokens de prueba | 0.106100 | 0.000432
150 tokens de prueba | 0.325295 | 0.000607
```

El script `analizador_benchmark.py` ejecuta ambos algoritmos sobre la misma cadena y mide el tiempo exacto en segundos mediante la librería `time`.

5. Análisis de Resultados (Evidencia del Benchmark)



Al ejecutar el programa en la terminal, los resultados muestran una brecha crítica de rendimiento:

- **Cadenas Cortas (10 tokens):** Ambos algoritmos responden en milisegundos.
- **Cadenas Largas (150 tokens):** Mientras que el motor tipo **Bison** mantiene un tiempo casi instantáneo (cerca de 0.001s), el algoritmo **CYK** muestra un retraso visible debido a la carga computacional de su matriz de estados.

Conclusión: La comparativa demuestra que para el desarrollo de compiladores modernos, el uso de gramáticas importadas con motores eficientes (LALR/Bison) es la única vía viable para manejar archivos de código fuente de gran tamaño.